Stage 2 Concierge Claiming — Implementation Report

Outcome

Stage 2 is implemented. Platform admins can create ownerless, menu-only vendors inside a pod, build and publish those vendors normally, and invite a specific verified account to claim the existing profile. Claim acceptance adds or promotes the claimant's `VendorMembership` to `owner` without creating another vendor or modifying the business's menu, availability, ordering intent, pod attachment, routing, or payment configuration.

The architecture preserves the central product rule: `Vendor` is a business entity and `VendorMembership` is user access. A vendor is claimed only when at least one owner-level membership exists.

Architecture implemented

Area	Implementation
Ownership source of truth	Derived from `VendorMembership.role === owner`; no `Vendor.ownerId` or persisted claimed flag
Claim invitation	Dedicated one-row-per-vendor `VendorClaimInvite` model
Token storage	32-byte base64url raw token delivered once; only its SHA-256 hash is stored
Token lifecycle	Seven-day expiry, rotation on resend, explicit revocation, atomic single-use consumption
Claim transaction	Serializable transaction with retry for write conflicts
Claimant requirements	Authenticated, enabled, verified account with normalized email matching the invitation
Concierge vendor defaults	Attached to selected pod, active profile, `orderingEnabled: false`, no user or membership
Authentication return	Safe internal `/claim/vendor/[token]` path survives login, registration, and email verification
Existing self-service flow	Unchanged; self-service vendor creation still creates an owner membership

`PodVendorInvite` was intentionally not reused. That model attaches an already-owned vendor to a pod and does not represent ownership transfer.

Schema and migration

Added `VendorClaimInvite` with:

- unique `vendorId`, ensuring one reusable lifecycle row per vendor;
- unique `tokenHash`;
- normalized invited email;
- expiry, claim, revocation, sent, created, and updated timestamps;
- nullable inviter and claimant user relations;
- vendor cascade deletion and user `SetNull` deletion behavior;
- indexes for email, expiry, inviter, and claimant.

Migration:

- `prisma/migrations/20260905033000_vendor_claim_invite/migration.sql`
- additive and non-destructive;
- does not update existing vendor, membership, menu, pod, ordering, routing, or payment rows;
- rollback is documented as dropping `VendorClaimInvite`.

The connected Supabase database was inspected read-only. The migration is pending and was not deployed because applying it would mutate the connected environment.

Admin concierge creation

The admin pod detail now includes a compact “Create vendor for this pod” workflow. It:

- validates and normalizes vendor/contact input;
- verifies the selected pod;
- detects an exact normalized-name duplicate in that pod;
- requires an explicit override for a legitimate duplicate;
- creates the vendor and active `PodVendor` transactionally;
- explicitly starts the vendor in menu-only mode;
- creates no user and no `VendorMembership`;
- writes no Stripe, Square, Deliverect, POS, or routing data;
- audits `UNCLAIMED_VENDOR_CREATED`;
- revalidates admin, pod, vendor, and public surfaces.

Claim lifecycle and security

Admins can send, resend, and revoke claim invitations from the vendor detail Ownership section.

Security controls include:

- cryptographically secure token generation;
- SHA-256 hash-only persistence;
- token rotation on resend, invalidating old links immediately;
- seven-day expiry;
- active/deleted/already-claimed vendor rejection;

- normalized invited-email validation;
- verified matching-account enforcement;
- no raw token in audit data;
- redacted transactional-email body previews, preventing bearer-link leakage in dry-run/log-only mode;
- admin-only server actions for create/send/resend/revoke;
- explicit claim action requiring authentication.

The claim transaction re-reads the user, invitation, and vendor; verifies lifecycle and email; checks that no owner exists; atomically consumes the pending invitation with `updateMany`; creates or promotes only the claimant's membership; and updates registration state. Serializable conflicts retry up to three attempts. Simultaneous claims yield one successful owner.

Claim and first-login experience

`/claim/vendor/[token]` provides:

- vendor and pod context;
- invalid, expired, revoked, and already-claimed states;
- login and registration links with a safe return path;
- invited-email display and wrong-account guidance;
- verification-email resend for matching unverified users;
- one explicit Claim action.

New registrations return to the claim page instead of being forced through vendor creation. The claim path is stored only as sanitized verification-token metadata and URL state. Successful verification displays “Continue to claim vendor.”

After acceptance, users are sent to `/vendor/[vendorId]/dashboard?claimed=1`, where one concise success message is shown. Existing membership authorization immediately grants access to the already-built menu, profile, and hours.

Admin visibility

- Vendor detail has a dedicated Ownership section with claim status, owner identity, and lifecycle controls.
- Pod roster shows a secondary ownership label and links to vendor admin.
- Vendor search includes a contained Claimed/Unclaimed filter.
- Claim state remains absent from customer-facing pod and menu surfaces.
- Unclaimed vendors remain eligible for normal public profile and menu readiness.

Data-preservation guardrails

Claim acceptance writes only:

- `VendorClaimInvite` claim lifecycle fields;

- one `VendorMembership` row or role promotion;
- claimant `User.registrationIntent` and `needsAccountRoleSelection`;
- `AdminAuditLog`.

Concierge creation writes only:

- `Vendor`;
- `PodVendor`;
- `AdminAuditLog`.

Tests assert that claim acceptance does not update `Vendor`, menu items/versions, pod membership, ordering mode, routing, Stripe, Square, or Deliverect data.

Validation results

Validation	Result
`npx prisma validate`	Passed
`npx prisma generate`	Passed
Stage 2 + auth regression suite	Passed: 8 files, 73 tests
Production `npm run build`	Passed; compilation, type validity, and 80 static pages completed
`git diff --check`	Passed; one informational LF-to-CRLF warning
`npx prisma migrate status`	Expected pending migration: `20260905033000_vendor_claim_invite`
Standalone `npx tsc --noEmit`	Existing baseline: 80 errors, none in Stage 2 files
Full repository test suite	Existing baseline: 453 files / 3,089 tests passed; 19 files / 32 tests failed; 12 unhandled mock errors

Focused coverage includes:

- owner-aware claim-state derivation, including staff-only access;
- duplicate warning and explicit override;
- no fake user/membership creation;
- secure hashing, expiry, rotation, revocation, and used-token behavior;
- wrong-account, disabled, unverified, expired, and independently claimed rejection;
- atomic consumption and simultaneous double-claim behavior;
- data-preservation assertions;
- admin authorization contracts;
- public invisibility of claim metadata;
- safe login/registration/verification return handling;

- post-claim menu authorization and dashboard success state.

The repository-wide failures are outside Stage 2. Representative baseline failures include stale Prisma mocks in pending-order reuse tests, pre-existing UI contract mismatches, payout test fixtures, Square OAuth route expectations, and operational API mocks.

Manual QA status

Code-level and production-build QA completed. A live browser walkthrough was not run because the connected database does not yet contain `VendorClaimInvite`; exercising the workflow would require deploying the pending migration to the shared Supabase environment. The implementation intentionally did not mutate that environment.

Recommended deployment smoke test after migration:

1. Create an unclaimed vendor from an admin pod.
2. Confirm it is public and menu-only with no owner membership.
3. Send, resend, and revoke invitations; verify old links fail.
4. Claim with a new verified account and an existing verified account.
5. Confirm a wrong account cannot claim.
6. Confirm exactly one owner after concurrent attempts.
7. Confirm existing menu, hours, profile, pod attachment, and ordering intent are unchanged.
8. Confirm immediate menu-builder/dashboard access and the one-time success message.

Principal files

- `prisma/schema.prisma`
- `prisma/migrations/20260905033000_vendor_claim_invite/migration.sql`
- `src/services/admin-concierge-vendor.service.ts`
- `src/services/vendor-claim-invite.service.ts`
- `src/lib/vendor-claim-state.ts`
- `src/actions/vendor-claim.actions.ts`
- `src/app/claim/vendor/[token]/page.tsx`
- `src/app/claim/vendor/[token]/VendorClaimPanel.tsx`
- `src/app/admin/(dashboard)/pods/[podId]/AdminPodOverview.tsx`
- `src/app/admin/(dashboard)/vendors/[vendorId]/AdminVendorOverview.tsx`
- `src/app/admin/(dashboard)/vendors/AdminVendorSearchForm.tsx`
- `src/services/email-verification.service.ts`
- `src/app/register/RegisterForm.tsx`

Deferred work

- Deploy the additive migration to the intended environment.

- Complete the live browser smoke test after deployment.
- Resolve repository-wide baseline type/test failures separately; none were introduced by Stage 2.